NARAWEAR ANALYTICS SPECIFICATION

Event Tracking, Tech Stack & MCP Implementation

Version: 1.0
Date: February 8, 2026
Tech Stack: React → Shopify → PostgreSQL → MCP Server
Campaign Platform: Meta Ads (Facebook/Instagram)

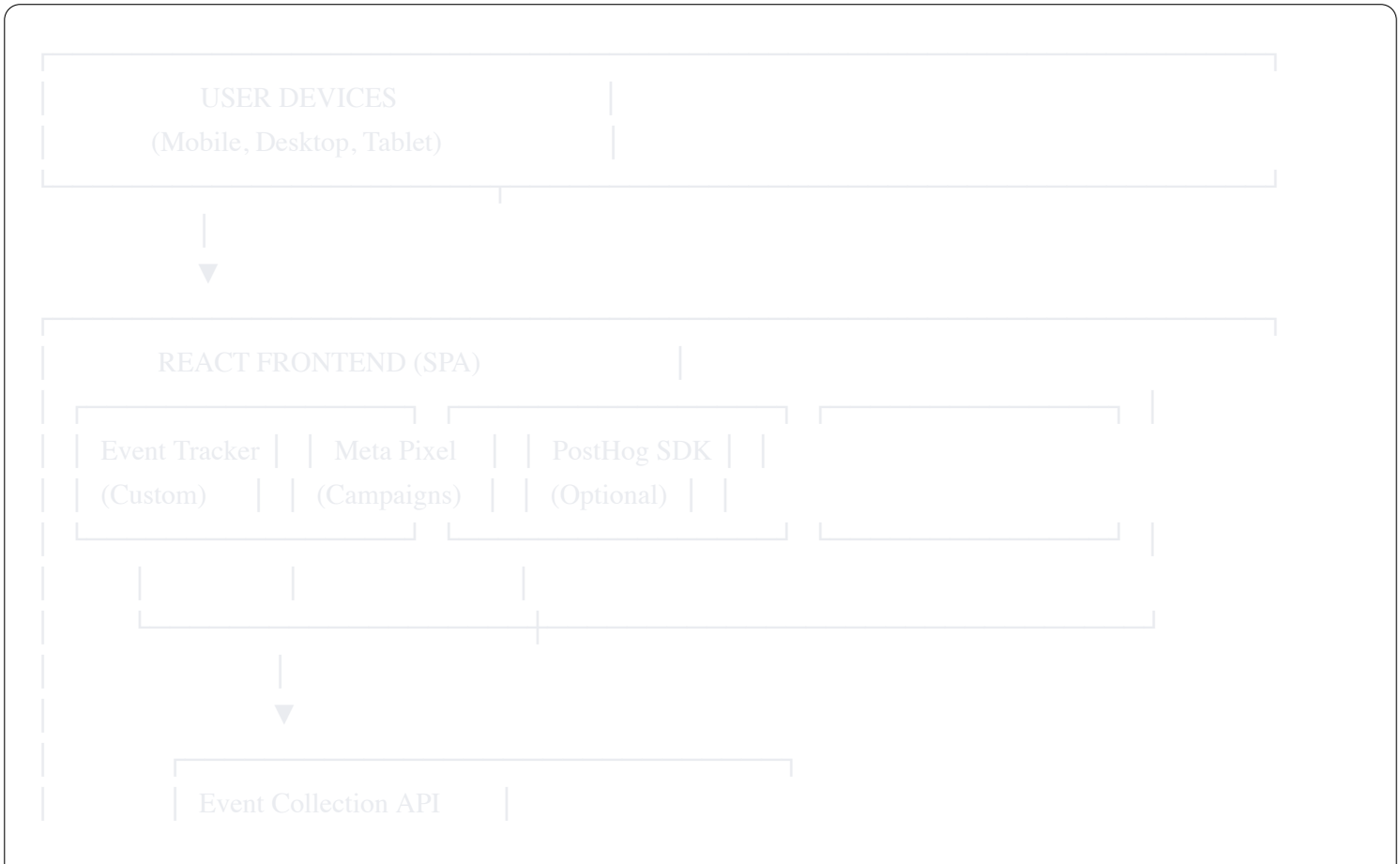
TABLE OF CONTENTS

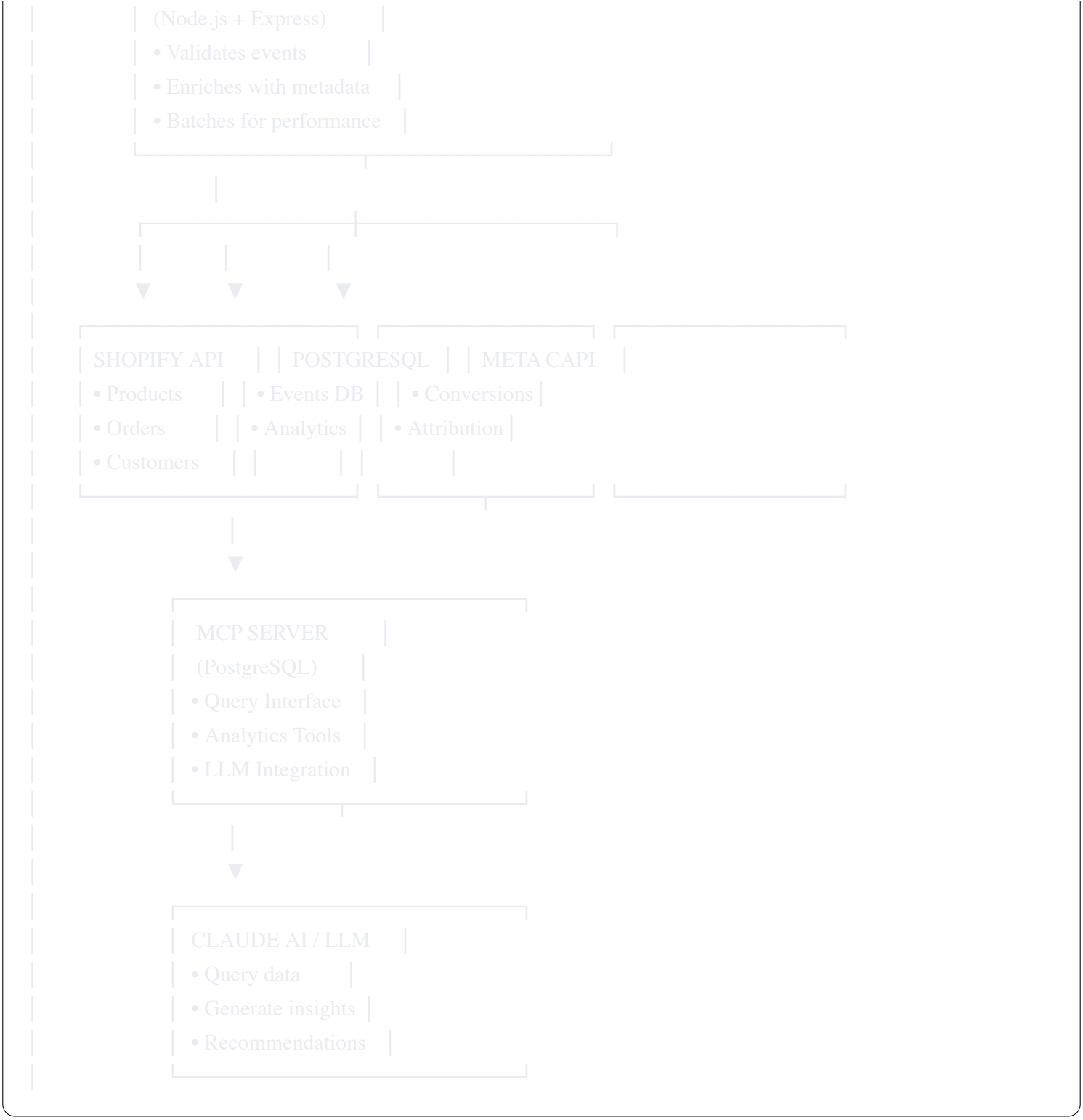
1. Tech Stack Architecture
2. Complete Event Catalog (Tabular)
3. Common Field Attributes
4. Database Schema
5. MCP Server Specification
6. Implementation Guide

---

1. TECH STACK ARCHITECTURE

System Flow Diagram





Technology Stack

Layer	Technology	Purpose	Why
Frontend	React 18+	SPA with event tracking	Modern, component-based
State Management	React Context / Zustand	Global state for cart, user	Lightweight
Event Tracking	Custom + Meta Pixel	Capture user events	Full control + Meta ads
Analytics (Optional)	PostHog	Session replay, funnels	Free tier, powerful
Backend Platform	Shopify (Headless)	E-commerce engine	Industry standard

Event API	Node.js + Express	Event collector/validator	Fast, JavaScript native
Message Queue	Redis (Optional)	Event buffering	Performance optimization
Database	PostgreSQL 15+	Event data warehouse	Powerful, MCP-compatible
MCP Server	TypeScript	AI query interface	Claude integration
Ad Platform	Meta Ads (FB/IG)	Campaigns	Target audience
Meta Integration	Conversions API	Server-side tracking	iOS 14+ accuracy
Hosting - Frontend	Vercel	React deployment	Fast CDN
Hosting - API	Railway / Render	API + PostgreSQL	Easy scaling

---

2. COMPLETE EVENT CATALOG

Event Summary Table

#	Event Name	Category	Trigger	Priority	Meta Pixel
1	`page_view`	Traffic	Every page load	Critical	✅ PageView
2	`session_start`	Traffic	First interaction	High	❌
3	`session_end`	Traffic	Session timeout/exit	Medium	❌
4	`collection_view`	Discovery	Collection page view	High	✅ ViewContent
5	`product_list_view`	Discovery	Products visible	High	❌
6	`product_click`	Discovery	Product clicked	High	❌
7	`product_view`	Discovery	Product page view	Critical	✅ ViewContent
8	`product_image_interaction`	Engagement	Image zoom/carousel	Medium	❌
9	`variant_selection`	Engagement	Size/color selected	Medium	❌
10	`scroll_depth`	Engagement	Scroll milestone	Low	❌
11	`element_visibility`	Engagement	Element in viewport	Low	❌
12	`content_interaction`	Engagement	Accordion/tab click	Medium	❌
13	`size_guide_view`	Engagement	Size guide opened	Medium	❌

14	sustainability_info_view	Engagement	Sustainability viewed	High	❌
15	review_interaction	Engagement	Review section used	Medium	❌
16	search_initiated	Search	Search started	High	✅ Search
17	search_results	Search	Results displayed	High	❌
18	search_no_results	Search	Zero results	High	❌
19	search_result_click	Search	Result clicked	High	❌
20	add_to_cart	Cart	Item added to cart	Critical	✅ AddToCart
21	remove_from_cart	Cart	Item removed	High	❌
22	cart_view	Cart	Cart opened/viewed	High	❌
23	cart_quantity_change	Cart	Quantity updated	Medium	❌
24	add_to_wishlist	Wishlist	Item wishlisted	Medium	✅ AddToWishlist
25	remove_from_wishlist	Wishlist	Item removed	Low	❌
26	promo_code_applied	Promotion	Discount applied	High	❌
27	promo_code_failed	Promotion	Invalid code	Medium	❌
28	promo_banner_click	Promotion	Banner clicked	Medium	❌
29	checkout_started	Checkout	Checkout initiated	Critical	✅ InitiateCheckout
30	checkout_step_viewed	Checkout	Step entered	High	❌
31	checkout_step_completed	Checkout	Step completed	High	❌
32	shipping_method_selected	Checkout	Shipping chosen	High	❌
33	payment_method_selected	Checkout	Payment chosen	High	✅ AddPaymentInfo
34	checkout_abandoned	Checkout	Checkout exited	Critical	❌
35	purchase	Purchase	Order completed	Critical	✅ Purchase
36	order_confirmation_view	Purchase	Thank you page	High	❌
37	repeat_purchase	Purchase	2nd+ purchase	High	❌
38	account_signup_initiated	Account	Signup started	High	❌
39	account_created	Account	Account created	High	✅

					CompleteRegistration
40	login	Account	User logged in	Medium	❌
41	logout	Account	User logged out	Low	❌
42	profile_update	Account	Profile edited	Low	❌
43	address_added	Account	Address added	Medium	❌
44	order_tracking_view	Post-Purchase	Tracking viewed	Medium	❌
45	review_prompt_viewed	Post-Purchase	Review request shown	Medium	❌
46	review_submitted	Post-Purchase	Review posted	High	❌
47	referral_link_shared	Post-Purchase	Referral shared	Medium	❌
48	newsletter_signup	Email	Newsletter subscribed	High	✅ Lead
49	newsletter_unsubscribe	Email	Unsubscribed	Medium	❌
50	email_clicked	Email	Email link clicked	Medium	❌
51	product_shared	Social	Product shared	Medium	❌
52	social_login	Social	Social auth used	Medium	❌
53	help_article_view	Support	FAQ viewed	Low	❌
54	live_chat_initiated	Support	Chat started	High	✅ Contact
55	contact_form_submitted	Support	Form submitted	High	✅ Contact
56	page_error	Technical	Error occurred	High	❌
57	form_abandonment	Technical	Form not completed	Medium	❌
58	filter_applied	Technical	Filter used	Medium	❌
59	sort_changed	Technical	Sort changed	Low	❌
60	first_visit	Onboarding	First site visit	High	❌
61	welcome_popup_interaction	Onboarding	Popup interaction	Medium	❌

62	sustainability_page_view	Sustainability	Sustainability page	High	❌
63	certification_click	Sustainability	Cert badge clicked	Medium	❌
64	material_source_viewed	Sustainability	Material info viewed	Medium	❌

---

3. COMMON FIELD ATTRIBUTES

3.1 Universal Fields (Present in ALL Events)

Field Name	Data Type	Required	Description	Example
event_id	UUID	Yes	Unique event identifier	550e8400-e29b-41d4-a716-446655440000
event_name	VARCHAR(50)	Yes	Event name	product_view
event_timestamp	TIMESTAMP	Yes	Event time (ISO 8601)	2026-02-08T14:30:00Z
session_id	VARCHAR(100)	Yes	Session identifier	sess_abc123xyz
user_id	VARCHAR(100)	No	Shopify customer ID (if logged in)	cust_7234892374
anonymous_id	VARCHAR(100)	Yes	Anonymous cookie ID	anon_9876543210

3.2 User Context Fields

Field Name	Data Type	Required	Description	Example
is_logged_in	BOOLEAN	Yes	Login status	true
customer_email	VARCHAR(255)	No	Customer email (if logged in)	user@example.com
customer_phone	VARCHAR(20)	No	Customer phone	+919876543210
is_returning_customer	BOOLEAN	No	Has previous orders	true
customer_ltv	DECIMAL(10,2)	No	Lifetime value	8500.00
previous_orders_count	INTEGER	No	Order count	3

3.3 Device & Browser Fields

Field Name	Data Type	Required	Description	Example	Possible Values
device_type	VARCHAR(20)	Yes	Device category	mobile	mobile, tablet, desktop
browser_name	VARCHAR(50)	Yes	Browser name	Chrome	Chrome, Safari, Firefox, Edge
browser_version	VARCHAR(20)	No	Browser version	120.0.6099	-
os_name	VARCHAR(50)	Yes	Operating system	iOS	iOS, Android, Windows, macOS
os_version	VARCHAR(20)	No	OS version	17.2	-
screen_resolution	VARCHAR(20)	No	Screen size	1920x1080	-
viewport_size	VARCHAR(20)	No	Viewport dimensions	1440x900	-

3.4 Traffic Source Fields (UTM Parameters)

Field Name	Data Type	Required	Description	Example
traffic_source	VARCHAR(50)	No	Traffic source type	organic
traffic_medium	VARCHAR(50)	No	Traffic medium	search
traffic_campaign	VARCHAR(200)	No	Campaign name	summer_sale_2026
utm_source	VARCHAR(200)	No	UTM source	instagram
utm_medium	VARCHAR(200)	No	UTM medium	social
utm_campaign	VARCHAR(200)	No	UTM campaign	influencer_march
utm_content	VARCHAR(200)	No	UTM content	story_swipe_up
utm_term	VARCHAR(200)	No	UTM term	sustainable_fashion

Possible Values for traffic_source: organic, paid, direct, referral, social, email

3.5 Geography Fields

Field Name	Data Type	Required	Description	Example
country	VARCHAR(2)	No	Country code (ISO 3166-1)	IN
state	VARCHAR(100)	No	State/province	Maharashtra
city	VARCHAR(100)	No	City	Mumbai
pincode	VARCHAR(20)	No	Postal code	410206
ip_address	VARCHAR(45)	No	Hashed IP	hash_192.168.1.1

3.6 Page Context Fields

Field Name	Data Type	Required	Description	Example	Possible Values
page_url	TEXT	Yes	Full page URL	https://narawear.com/...	-
page_path	VARCHAR(500)	Yes	URL path	/products/dress	-
page_title	VARCHAR(200)	Yes	Page title	Organic Cotton Dress	-
page_type	VARCHAR(50)	Yes	Page category	product	home, collection, product, cart, checkout, account
page_referrer	TEXT	No	Previous page URL	https://google.com/...	-

3.7 Product Fields

Field Name	Data Type	Required	Description	Example
product_id	VARCHAR(100)	Yes	Shopify product ID	prod_8472639847
variant_id	VARCHAR(100)	Yes	Shopify variant ID	var_3847562938
product_name	VARCHAR(300)	Yes	Product name	Organic Cotton Dress
product_handle	VARCHAR(200)	Yes	URL slug	organic-cotton-dress

product_handle	VARCHAR(200)	Yes	URL slug	organic-cotton-dress
sku	VARCHAR(100)	Yes	Stock keeping unit	OCD-BLU-M
category	VARCHAR(100)	No	Product category	Dresses
subcategory	VARCHAR(100)	No	Product subcategory	Maxi Dresses
collection	VARCHAR(200)	No	Collection name	Summer Collection
product_type	VARCHAR(100)	No	Shopify product type	Dress
vendor	VARCHAR(100)	No	Brand/vendor	NaraWear
color	VARCHAR(50)	No	Color variant	Blue
size	VARCHAR(20)	No	Size variant	M
material	VARCHAR(100)	No	Material type	Organic Cotton
sustainability_tag	VARCHAR(100)	No	Sustainability label	Eco-Friendly

3.8 Pricing Fields

Field Name	Data Type	Required	Description	Example
price	DECIMAL(10,2)	Yes	Current price	1999.00
compare_at_price	DECIMAL(10,2)	No	Original price (if on sale)	2499.00
discount_amount	DECIMAL(10,2)	No	Discount value	500.00
discount_percent	INTEGER	No	Discount percentage	20
currency	VARCHAR(3)	Yes	Currency code (ISO 4217)	INR

3.9 Inventory Fields

Field Name	Data Type	Required	Description	Example	Possible Values
in_stock	BOOLEAN	Yes	Stock availability	true	true, false
stock_quantity	INTEGER	No	Available quantity	15	-
inventory_policy	VARCHAR(20)	No	Out-of-stock behavior	deny	continue, deny

3.10 Cart/Checkout Fields

Field Name	Data Type	Required	Description	Example
cart_id	VARCHAR(100)	No	Cart token	cart_9283746529
checkout_id	VARCHAR(100)	No	Checkout token	chk_9283746529
items_count	INTEGER	Yes	Total items	2
unique_products_count	INTEGER	Yes	Unique products	2
subtotal	DECIMAL(10,2)	Yes	Subtotal	3998.00
tax_amount	DECIMAL(10,2)	No	Tax	0.00
shipping_amount	DECIMAL(10,2)	No	Shipping cost	100.00
discount_amount	DECIMAL(10,2)	No	Total discount	500.00
total_value	DECIMAL(10,2)	Yes	Final total	3598.00

3.11 Order Fields

Field Name	Data Type	Required	Description	Example
order_id	VARCHAR(100)	Yes	Shopify order ID	ord_2938475629
order_number	VARCHAR(50)	Yes	Human-readable order #	#1001
order_status	VARCHAR(50)	No	Order status	paid

Possible Values for order_status: pending, paid, authorized, partially_fulfilled, fulfilled, cancelled, refunded

3.12 Items Array (JSONB for Cart/Checkout Events)

Structure:

```
json
```

```
[
  {
    "line_id": "line_1",
    "product_id": "prod_8472639847",
    "variant_id": "var_3847562938",
    "product_name": "Organic Cotton Dress",
    "sku": "OCD-BLU-M",
    "category": "Dresses",
    "price": 1999.00,
    "quantity": 1,
    "line_total": 1999.00,
    "color": "Blue",
    "size": "M"
  }
]
```

---

4. DATABASE SCHEMA

4.1 Main Events Table

```sql
CREATE TABLE events (
    -- Primary Key
    event_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    -- Event Core
    event_name VARCHAR(50) NOT NULL,
    event_timestamp TIMESTAMP NOT NULL DEFAULT NOW(),
    event_category VARCHAR(50), -- traffic, discovery, cart, checkout, etc.

    -- Session & User
    session_id VARCHAR(100) NOT NULL,
    user_id VARCHAR(100), -- NULL for anonymous users
    anonymous_id VARCHAR(100) NOT NULL,
    is_logged_in BOOLEAN DEFAULT FALSE,

    -- Customer Details
    customer_email VARCHAR(255),
    customer_phone VARCHAR(20),
    is_returning_customer BOOLEAN,
    customer_ltv DECIMAL(10,2),
    previous_orders_count INTEGER,
```

```sql
    -- Device & Browser
    device_type VARCHAR(20),
    browser_name VARCHAR(50),
    browser_version VARCHAR(20),
    os_name VARCHAR(50),
    os_version VARCHAR(20),
    screen_resolution VARCHAR(20),
    viewport_size VARCHAR(20),

    -- Traffic Source
    traffic_source VARCHAR(50),
    traffic_medium VARCHAR(50),
    traffic_campaign VARCHAR(200),
    utm_source VARCHAR(200),
    utm_medium VARCHAR(200),
    utm_campaign VARCHAR(200),
    utm_content VARCHAR(200),
    utm_term VARCHAR(200),

    -- Geography
    country VARCHAR(2),
    state VARCHAR(100),
    city VARCHAR(100),
    pincode VARCHAR(20),
    ip_address VARCHAR(45), -- Hashed for privacy

    -- Page Context
    page_url TEXT,
    page_path VARCHAR(500),
    page_title VARCHAR(200),
    page_type VARCHAR(50),
    page_referrer TEXT,

    -- Event-Specific Data (JSONB for flexibility)
    event_properties JSONB,

    -- Meta Tracking
    sent_to_meta_pixel BOOLEAN DEFAULT FALSE,
    meta_event_id VARCHAR(100), -- For deduplication

    -- Indexes
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Indexes for Performance
CREATE INDEX idx_events_timestamp ON events(event_timestamp DESC);
```

```sql
CREATE INDEX idx_events_timestamp ON events(event_timestamp DESC);
CREATE INDEX idx_events_session ON events(session_id);
CREATE INDEX idx_events_user ON events(user_id);
CREATE INDEX idx_events_name ON events(event_name);
CREATE INDEX idx_events_category ON events(event_category);

CREATE INDEX idx_events_properties ON events USING GIN(event_properties);
```

4.2 Products Table (Synced from Shopify)

```sql
CREATE TABLE products (
    product_id VARCHAR(100) PRIMARY KEY,
    product_name VARCHAR(300) NOT NULL,
    product_handle VARCHAR(200) UNIQUE NOT NULL,
    product_type VARCHAR(100),
    vendor VARCHAR(100),
    category VARCHAR(100),
    subcategory VARCHAR(100),
    collections TEXT[], -- Array of collection names
    material VARCHAR(100),
    sustainability_tags TEXT[],
    certifications TEXT[],

    -- Pricing
    price DECIMAL(10,2),
    compare_at_price DECIMAL(10,2),

    -- Inventory
    in_stock BOOLEAN DEFAULT TRUE,
    total_inventory INTEGER,

    -- Metadata
    images TEXT[],
    description TEXT,
    tags TEXT[],

    -- Shopify Sync
    shopify_created_at TIMESTAMP,
    shopify_updated_at TIMESTAMP,
    synced_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_products_handle ON products(product_handle);
CREATE INDEX idx_products_category ON products(category);
```

4.3 Product Variants Table

```sql
CREATE TABLE product_variants (
    variant_id VARCHAR(100) PRIMARY KEY,
    product_id VARCHAR(100) REFERENCES products(product_id),
    variant_title VARCHAR(200),
    sku VARCHAR(100) UNIQUE,

    -- Variant Options
    color VARCHAR(50),
    size VARCHAR(20),
    style VARCHAR(50),

    -- Pricing
    price DECIMAL(10,2),
    compare_at_price DECIMAL(10,2),

    -- Inventory
    in_stock BOOLEAN DEFAULT TRUE,
    inventory_quantity INTEGER,
    inventory_policy VARCHAR(20), -- continue or deny

    -- Metadata
    image_url TEXT,
    weight_grams INTEGER,

    synced_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_variants_product ON product_variants(product_id);
CREATE INDEX idx_variants_sku ON product_variants(sku);
```

4.4 Orders Table (Synced from Shopify)

```sql
CREATE TABLE orders (
    order_id VARCHAR(100) PRIMARY KEY,
    order_number VARCHAR(50) UNIQUE NOT NULL,
    checkout_id VARCHAR(100),

    -- Customer
```

```sql
    -- Customer
    customer_id VARCHAR(100),
    customer_email VARCHAR(255),
    customer_phone VARCHAR(20),

    -- Order Details
    order_status VARCHAR(50),
    financial_status VARCHAR(50),
    fulfillment_status VARCHAR(50),

    -- Amounts
    subtotal DECIMAL(10,2),
    tax_amount DECIMAL(10,2),
    shipping_amount DECIMAL(10,2),
    discount_amount DECIMAL(10,2),
    total_amount DECIMAL(10,2),
    currency VARCHAR(3),

    -- Items
    items_count INTEGER,
    line_items JSONB, -- Array of order items

    -- Discounts
    discount_codes TEXT[],

    -- Shipping
    shipping_method VARCHAR(100),
    shipping_address JSONB,

    -- Payment
    payment_method VARCHAR(50),
    payment_gateway VARCHAR(100),

    -- Attribution
    utm_source VARCHAR(200),
    utm_medium VARCHAR(200),
    utm_campaign VARCHAR(200),
    referring_site TEXT,

    -- Timestamps
    order_created_at TIMESTAMP,
    order_updated_at TIMESTAMP,
    synced_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_orders_customer ON orders(customer_id);
CREATE INDEX idx_orders_created ON orders(order_created_at DESC);
CREATE INDEX idx_orders_status ON orders(order_status);
```

4.5 Sessions Table (Aggregated)

sql

```sql
CREATE TABLE sessions (
    session_id VARCHAR(100) PRIMARY KEY,
    user_id VARCHAR(100),
    anonymous_id VARCHAR(100),

    -- Session Details
    session_start TIMESTAMP NOT NULL,
    session_end TIMESTAMP,
    session_duration_seconds INTEGER,

    -- Entry
    landing_page TEXT,
    entry_source VARCHAR(50),
    referrer_domain VARCHAR(200),

    -- UTM Parameters
    utm_source VARCHAR(200),
    utm_medium VARCHAR(200),
    utm_campaign VARCHAR(200),
    utm_content VARCHAR(200),
    utm_term VARCHAR(200),

    -- Device
    device_type VARCHAR(20),
    browser_name VARCHAR(50),
    os_name VARCHAR(50),

    -- Geography
    country VARCHAR(2),
    state VARCHAR(100),
    city VARCHAR(100),

    -- Activity
    pages_viewed INTEGER DEFAULT 0,
    events_count INTEGER DEFAULT 0,
    products_viewed INTEGER DEFAULT 0,

    -- Conversion
    converted BOOLEAN DEFAULT FALSE,
    order_id VARCHAR(100),
```

```sql
    revenue DECIMAL(10,2),

    -- Cart
    cart_value_at_exit DECIMAL(10,2),
    items_in_cart_at_exit INTEGER,

    created_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_sessions_user ON sessions(user_id);
CREATE INDEX idx_sessions_start ON sessions(session_start DESC);
CREATE INDEX idx_sessions_converted ON sessions(converted);
```

4.6 Customers Table (Synced from Shopify)

```sql
CREATE TABLE customers (
    customer_id VARCHAR(100) PRIMARY KEY,
    email VARCHAR(255) UNIQUE,
    phone VARCHAR(20),
    first_name VARCHAR(100),
    last_name VARCHAR(100),

    -- Status
    accepts_marketing BOOLEAN,
    marketing_opt_in_level VARCHAR(50),

    -- Stats
    orders_count INTEGER DEFAULT 0,
    total_spent DECIMAL(10,2) DEFAULT 0,
    average_order_value DECIMAL(10,2),

    -- Segments
    customer_segment VARCHAR(50), -- new, active, at_risk, churned
    customer_ltv DECIMAL(10,2),

    -- Dates
    first_order_date TIMESTAMP,
    last_order_date TIMESTAMP,
    created_at TIMESTAMP,
    updated_at TIMESTAMP,

    -- Shopify
    shopify_created_at TIMESTAMP,
    synced_at TIMESTAMP DEFAULT NOW()
```

```
);

CREATE INDEX idx_customers_email ON customers(email);
CREATE INDEX idx_customers_segment ON customers(customer_segment);
```

---

5. MCP SERVER SPECIFICATION

5.1 What is MCP?

Model Context Protocol (MCP) is an open protocol that standardizes how applications provide context to LLMs (like Claude). It allows Claude to query your PostgreSQL database directly and generate insights.

5.2 MCP Server Architecture

```
┌─────────────────────────────────────────────────────────────┐
│                        CLAUDE AI                            │
│  "Show me conversion rate by traffic source"                │
└─────────────────────────────────────────────────────────────┘
                          │
                          │ MCP Protocol
                          ▼
┌─────────────────────────────────────────────────────────────┐
│                   MCP SERVER (TypeScript/Node.js)           │
│                                                             │
│  ┌─────────────────────────────────────────────────────┐    │
│  │ MCP Tools (Available to Claude)                     │    │
│  │ • query_events                                      │    │
│  │ • get_conversion_funnel                             │    │
│  │ • get_product_performance                           │    │
│  │ • get_customer_segments                             │    │
│  │ • get_session_analysis                              │    │
│  │ • get_attribution_report                            │    │
│  └─────────────────────────────────────────────────────┘    │
└─────────────────────────────────────────────────────────────┘
                          │
                          │ SQL Queries
                          ▼
┌─────────────────────────────────────────────────────────────┐
│                     POSTGRESQL DATABASE                     │
│  • events table                                             │
│  • sessions table                                           │
│  • orders table                                             │
│  • products table                                           │
└─────────────────────────────────────────────────────────────┘
```

5.3 MCP Server Implementation

Directory Structure:

```
narawear-mcp-server/
├── package.json
├── tsconfig.json
├── src/
│   ├── index.ts        # Main MCP server
│   ├── db.ts           # PostgreSQL connection
│   ├── tools/
│   │   ├── query_events.ts
│   │   ├── conversion_funnel.ts
│   │   ├── product_performance.ts
│   │   ├── customer_segments.ts
│   │   ├── session_analysis.ts
│   │   └── attribution.ts
│   └── types.ts        # TypeScript types
└── README.md
```

---

package.json:

```json
{
  "name": "narawear-mcp-server",
  "version": "1.0.0",
  "type": "module",
  "bin": {
    "narawear-mcp": "./build/index.js"
  },
  "scripts": {
    "build": "tsc",
    "start": "node build/index.js"
  },
  "dependencies": {
    "@modelcontextprotocol/sdk": "^0.5.0",
    "pg": "^8.11.3"
  },
  "devDependencies": {
    "@types/node": "^20.10.0",
    "@types/pg": "^8.10.0",
    "typescript": "^5.3.0"
  }
}
```

src/index.ts:

```typescript
#!/usr/bin/env node
import { Server } from "@modelcontextprotocol/sdk/server/index.js";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";
import {
  CallToolRequestSchema,
  ListToolsRequestSchema,
} from "@modelcontextprotocol/sdk/types.js";
import { getDbPool } from "./db.js";
import { queryEventsHandler } from "./tools/query_events.js";
import { conversionFunnelHandler } from "./tools/conversion_funnel.js";
import { productPerformanceHandler } from "./tools/product_performance.js";
import { customerSegmentsHandler } from "./tools/customer_segments.js";
import { sessionAnalysisHandler } from "./tools/session_analysis.js";
import { attributionReportHandler } from "./tools/attribution.js";

const server = new Server(
  {
    name: "narawear-analytics",
    version: "1.0.0",
  },
  {
    capabilities: {
      tools: {},
    },
  }
);

// List available tools
server.setRequestHandler(ListToolsRequestSchema, async () => {
  return {
    tools: [
      {
        name: "query_events",
        description:
          "Query raw events from the database with flexible filtering. Use for custom analysis.",
        inputSchema: {
          type: "object",
          properties: {
            event_name: {
              type: "string",
              description: "Filter by event name (e.g., 'product_view', 'add_to_cart')",
            },
```

```
      start_date: {
        type: "string",
        description: "Start date (ISO format: YYYY-MM-DD)",
      },
      end_date: {
        type: "string",
        description: "End date (ISO format: YYYY-MM-DD)",
      },

      limit: {
        type: "number",
        description: "Max number of results (default: 100)",
      },
    },
  },
},
{
  name: "get_conversion_funnel",
  description:
    "Get conversion funnel analysis showing drop-off at each stage (product_view -> add_to_cart -> checkout -> purch
  inputSchema: {
    type: "object",
    properties: {
      start_date: { type: "string" },
      end_date: { type: "string" },
    },
    required: ["start_date", "end_date"],
  },
},
{
  name: "get_product_performance",
  description:
    "Get product performance metrics (views, add to cart rate, conversion rate, revenue)",
  inputSchema: {
    type: "object",
    properties: {
      start_date: { type: "string" },
      end_date: { type: "string" },
      limit: { type: "number", description: "Top N products (default: 20)" },
    },
    required: ["start_date", "end_date"],
  },
},
{
  name: "get_customer_segments",
  description:
    "Get customer segmentation (new, active, at-risk, churned) with metrics",
  inputSchema: {
```

```
        type: "object",
        properties: {},
      },
    },
    {
      name: "get_session_analysis",
      description:
        "Get session-level metrics (avg duration, pages per session, conversion rate)",

      inputSchema: {
        type: "object",
        properties: {
          start_date: { type: "string" },
          end_date: { type: "string" },
        },
        required: ["start_date", "end_date"],
      },
    },
    {
      name: "get_attribution_report",
      description:
        "Get attribution report showing revenue by traffic source/medium/campaign",
      inputSchema: {
        type: "object",
        properties: {
          start_date: { type: "string" },
          end_date: { type: "string" },
          group_by: {
            type: "string",
            enum: ["source", "medium", "campaign"],
            description: "Group results by source, medium, or campaign",
          },
        },
        required: ["start_date", "end_date", "group_by"],
      },
    },
  ],
};
});

// Handle tool calls
server.setRequestHandler(CallToolRequestSchema, async (request) => {
  const { name, arguments: args } = request.params;

  try {
    switch (name) {
      case "query_events":
        return await queryEventsHandler(args);
```

```
      case "get_conversion_funnel":
        return await conversionFunnelHandler(args);
      case "get_product_performance":
        return await productPerformanceHandler(args);
      case "get_customer_segments":
        return await customerSegmentsHandler(args);
      case "get_session_analysis":
        return await sessionAnalysisHandler(args);

      case "get_attribution_report":
        return await attributionReportHandler(args);
      default:
        throw new Error(`Unknown tool: ${name}`);
    }
  } catch (error: any) {
    return {
      content: [
        {
          type: "text",
          text: `Error executing ${name}: ${error.message}`,
        },
      ],
    };
  }
});

// Start server
async function main() {
  const transport = new StdioServerTransport();
  await server.connect(transport);
  console.error("NaraWear Analytics MCP Server running on stdio");
}

main().catch((error) => {
  console.error("Fatal error:", error);
  process.exit(1);
});
```

---

src/db.ts:

```typescript
```

```typescript
import pg from "pg";
const { Pool } = pg;

let pool: pg.Pool | null = null;

export function getDbPool(): pg.Pool {
  if (!pool) {
    pool = new Pool({
      host: process.env.DB_HOST || "localhost",
      port: parseInt(process.env.DB_PORT || "5432"),
      database: process.env.DB_NAME || "narawear_analytics",
      user: process.env.DB_USER || "postgres",
      password: process.env.DB_PASSWORD,
      max: 20,
      idleTimeoutMillis: 30000,
      connectionTimeoutMillis: 2000,
    });
  }
  return pool;
}
```

---

src/tools/query_events.ts:

```typescript
import { getDbPool } from "../db.js";

export async function queryEventsHandler(args: any) {
  const pool = getDbPool();

  const { event_name, start_date, end_date, limit = 100 } = args;

  let query = `
    SELECT
      event_id,
      event_name,
```

```
    event_timestamp,
    session_id,
    user_id,
    page_url,
    device_type,
    event_properties
  FROM events
  WHERE 1=1
`;

const params: any[] = [];
let paramIndex = 1;

if (event_name) {
  query += ` AND event_name = $${paramIndex}`;
  params.push(event_name);
  paramIndex++;
}

if (start_date) {
  query += ` AND event_timestamp >= $${paramIndex}::timestamp`;
  params.push(start_date);
  paramIndex++;
}

if (end_date) {
  query += ` AND event_timestamp <= $${paramIndex}::timestamp`;
  params.push(end_date);
  paramIndex++;
}

query += ` ORDER BY event_timestamp DESC LIMIT $${paramIndex}`;
params.push(limit);

const result = await pool.query(query, params);

return {
  content: [
    {
      type: "text",
      text: JSON.stringify(result.rows, null, 2),
    },
  ],
};
}
```

src/tools/conversion_funnel.ts:

typescript

```typescript
import { getDbPool } from "../db.js";

export async function conversionFunnelHandler(args: any) {
  const pool = getDbPool();
  const { start_date, end_date } = args;

  const query = `
    WITH funnel_data AS (
      SELECT
        session_id,
        MAX(CASE WHEN event_name = 'product_view' THEN 1 ELSE 0 END) as viewed,
        MAX(CASE WHEN event_name = 'add_to_cart' THEN 1 ELSE 0 END) as added_cart,
        MAX(CASE WHEN event_name = 'checkout_started' THEN 1 ELSE 0 END) as checkout,
        MAX(CASE WHEN event_name = 'purchase' THEN 1 ELSE 0 END) as purchased
      FROM events
      WHERE event_timestamp >= $1::timestamp
        AND event_timestamp <= $2::timestamp
      GROUP BY session_id
    )
    SELECT
      SUM(viewed) as product_views,
      SUM(added_cart) as add_to_carts,
      SUM(checkout) as checkouts,
```

```
    SUM(purchased) as purchases,
    ROUND(100.0 * SUM(added_cart) / NULLIF(SUM(viewed), 0), 2) as cart_rate,
    ROUND(100.0 * SUM(checkout) / NULLIF(SUM(added_cart), 0), 2) as checkout_rate,
    ROUND(100.0 * SUM(purchased) / NULLIF(SUM(checkout), 0), 2) as purchase_rate,
    ROUND(100.0 * SUM(purchased) / NULLIF(SUM(viewed), 0), 2) as overall_conversion
  FROM funnel_data
`;

const result = await pool.query(query, [start_date, end_date]);

return {
  content: [
    {
      type: "text",
      text: JSON.stringify(result.rows[0], null, 2),
    },
  ],
};
}
```

---

src/tools/product_performance.ts:

```typescript
import { getDbPool } from "../db.js";
```

```
export async function productPerformanceHandler(args: any) {
  const pool = getDbPool();
  const { start_date, end_date, limit = 20 } = args;

  const query = `
    SELECT
      p.product_id,
      p.product_name,
      p.category,
      COUNT(DISTINCT CASE WHEN e.event_name = 'product_view' THEN e.session_id END) as views,
      COUNT(DISTINCT CASE WHEN e.event_name = 'add_to_cart' THEN e.session_id END) as add_to_carts,
      COUNT(DISTINCT CASE WHEN e.event_name = 'purchase' THEN e.session_id END) as purchases,
      ROUND(
        100.0 * COUNT(DISTINCT CASE WHEN e.event_name = 'add_to_cart' THEN e.session_id END) /
        NULLIF(COUNT(DISTINCT CASE WHEN e.event_name = 'product_view' THEN e.session_id END), 0),
        2
      ) as add_to_cart_rate,
      COALESCE(SUM((e.event_properties->>'total_value')::numeric), 0) as revenue
    FROM products p
    LEFT JOIN events e ON
      e.event_properties->>'product_id' = p.product_id
      AND e.event_timestamp >= $1::timestamp
      AND e.event_timestamp <= $2::timestamp
    GROUP BY p.product_id, p.product_name, p.category
    ORDER BY views DESC
    LIMIT $3
  `;

  const result = await pool.query(query, [start_date, end_date, limit]);

  return {
    content: [
      {
        type: "text",
        text: JSON.stringify(result.rows, null, 2),
      },
    ],
  };
}
```

5.4 MCP Server Configuration for Claude Desktop

File: `~/Library/Application Support/Claude/claude_desktop_config.json` (macOS)

```
json
```

```
{
  "mcpServers": {
    "narawear-analytics": {
      "command": "node",
      "args": ["/path/to/narawear-mcp-server/build/index.js"],
      "env": {
        "DB_HOST": "localhost",
        "DB_PORT": "5432",
        "DB_NAME": "narawear_analytics",
        "DB_USER": "postgres",
        "DB_PASSWORD": "your_password"
      }
    }
  }
}
```

---

6. IMPLEMENTATION GUIDE

Phase 1: Frontend Event Tracking (Week 1-2)

6.1 Install Dependencies

```bash
npm install uuid
```

6.2 Create Event Tracker Hook

src/hooks/useEventTracker.js:

```javascript
import { useCallback, useEffect } from 'react';
import { v4 as uuidv4 } from 'uuid';

const EVENT_API_URL = process.env.REACT_APP_EVENT_API_URL;

// Get or create session ID
const getSessionId = () => {
  let sessionId = sessionStorage.getItem('session_id');
  if (!sessionId) {
    sessionId = `sess_${uuidv4()}`;
    sessionStorage.setItem('session_id', sessionId);
  }
  return sessionId;
};
```

```
// Get or create anonymous ID
const getAnonymousId = () => {
  let anonId = localStorage.getItem('anonymous_id');
  if (!anonId) {
    anonId = `anon_${uuidv4()}`;
    localStorage.setItem('anonymous_id', anonId);
  }
  return anonId;
};

// Get device info
const getDeviceInfo = () => {
  const ua = navigator.userAgent;
  return {
    device_type: /mobile/i.test(ua) ? 'mobile' : /tablet/i.test(ua) ? 'tablet' : 'desktop',
    browser_name: ua.includes('Chrome') ? 'Chrome' : ua.includes('Safari') ? 'Safari' : 'Other',
    os_name: ua.includes('Mac') ? 'macOS' : ua.includes('Win') ? 'Windows' : ua.includes('Android') ? 'Android' : ua.includ

    screen_resolution: `${window.screen.width}x${window.screen.height}`,
    viewport_size: `${window.innerWidth}x${window.innerHeight}`,
  };
};

// Extract UTM parameters
const getUTMParams = () => {
  const urlParams = new URLSearchParams(window.location.search);
  return {
    utm_source: urlParams.get('utm_source'),
    utm_medium: urlParams.get('utm_medium'),
    utm_campaign: urlParams.get('utm_campaign'),
    utm_content: urlParams.get('utm_content'),
    utm_term: urlParams.get('utm_term'),
  };
};

export const useEventTracker = () => {
  const trackEvent = useCallback(async (eventName, properties = {}) => {
    const event = {
      event_id: uuidv4(),
      event_name: eventName,
      event_timestamp: new Date().toISOString(),
      session_id: getSessionId(),
      anonymous_id: getAnonymousId(),
      user_id: properties.user_id || null,

      // Page context
      page_url: window.location.href,
      page_path: window.location.pathname,
```

```
    page_title: document.title,
    page_referrer: document.referrer,

    // Device info
    ...getDeviceInfo(),

    // UTM parameters
    ...getUTMParams(),

    // Custom properties
    ...properties,
  };

  // Send to your API
  try {
    await fetch(`${EVENT_API_URL}/events`, {
      method: 'POST',

      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(event),
    });

    // Also send to Meta Pixel if applicable
    if (window.fbq && shouldSendToMeta(eventName)) {
      sendToMetaPixel(eventName, properties);
    }
  } catch (error) {
    console.error('Error tracking event:', error);
  }
}, []);

return { trackEvent };
};

// Helper: Check if event should go to Meta Pixel
const shouldSendToMeta = (eventName) => {
  const metaEvents = [
    'page_view',
    'product_view',
    'add_to_cart',
    'checkout_started',
    'purchase',
    'search_initiated',
    'add_to_wishlist',
    'account_created',
    'newsletter_signup',
  ];
  return metaEvents.includes(eventName);
```

```
};

// Helper: Send to Meta Pixel
const sendToMetaPixel = (eventName, properties) => {
  const metaEventMap = {
    page_view: 'PageView',
    product_view: 'ViewContent',
    add_to_cart: 'AddToCart',
    checkout_started: 'InitiateCheckout',
    purchase: 'Purchase',
    search_initiated: 'Search',
    add_to_wishlist: 'AddToWishlist',
    account_created: 'CompleteRegistration',
    newsletter_signup: 'Lead',
  };

  const metaEvent = metaEventMap[eventName];

  if (metaEvent && window.fbq) {
    window.fbq('track', metaEvent, {
      content_ids: properties.product_id ? [properties.product_id] : [],
      content_type: 'product',
      value: properties.price || properties.total_value || 0,
      currency: properties.currency || 'INR',
    });
  }
};
```

6.3 Track Page Views

src/App.js:

```javascript
import { useEffect } from 'react';
import { useLocation } from 'react-router-dom';
import { useEventTracker } from './hooks/useEventTracker';

function App() {
  const location = useLocation();
  const { trackEvent } = useEventTracker();

  useEffect(() => {
    trackEvent('page_view', {
      page_type: getPageType(location.pathname),
    });
```

```javascript
  }, [location, trackEvent]);

  return (
    // Your app
  );
}

const getPageType = (pathname) => {
  if (pathname === '/') return 'home';
  if (pathname.includes('/collections')) return 'collection';
  if (pathname.includes('/products')) return 'product';
  if (pathname.includes('/cart')) return 'cart';
  if (pathname.includes('/checkout')) return 'checkout';
  return 'other';
};
```

6.4 Track Product View

src/pages/ProductPage.js:

```javascript
import { useEffect } from 'react';
import { useEventTracker } from '../hooks/useEventTracker';

function ProductPage({ product }) {
  const { trackEvent } = useEventTracker();

  useEffect(() => {
    if (product) {
      trackEvent('product_view', {
        product_id: product.id,
        product_name: product.title,
        product_handle: product.handle,
        price: product.price,
        category: product.category,
        in_stock: product.available,
      });
    }
  }, [product, trackEvent]);

  return (
    // Product page UI
  );
}
```

6.5 Track Add to Cart

src/components/AddToCartButton.js:

```javascript
import { useEventTracker } from '../hooks/useEventTracker';

function AddToCartButton({ product, variant, quantity }) {
  const { trackEvent } = useEventTracker();

  const handleAddToCart = () => {
    // Add to cart logic
    addToCart(variant.id, quantity);

    // Track event
    trackEvent('add_to_cart', {
      product_id: product.id,
      variant_id: variant.id,
      product_name: product.title,
      sku: variant.sku,
      price: variant.price,
      quantity: quantity,
      category: product.category,
      color: variant.color,
      size: variant.size,
      add_source: 'product_page',
    });
  };

  return <button onClick={handleAddToCart}>Add to Cart</button>;
}
```

Phase 2: Backend Event Collection API (Week 2-3)

6.6 Create Event API

backend/server.js:

```javascript
import express from 'express';
import cors from 'cors';
import pg from 'pg';
import crypto from 'crypto';

const app = express();
const { Pool } = pg;

app.use(cors());
app.use(express.json());

// PostgreSQL connection
const pool = new Pool({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  database: process.env.DB_NAME,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
});

// Event ingestion endpoint
app.post('/events', async (req, res) => {
  try {
    const event = req.body;


    // Enrich with server-side data
    event.ip_address = hashIP(req.ip);

    // Insert into database
    await pool.query(
      `INSERT INTO events (
        event_id, event_name, event_timestamp, session_id, user_id, anonymous_id,
        page_url, page_path, page_title, page_referrer,
        device_type, browser_name, os_name,
        utm_source, utm_medium, utm_campaign,
        event_properties
      ) VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12, $13, $14, $15, $16, $17)`,
      [
        event.event_id,
```

```
      event.event_name,
      event.event_timestamp,
      event.session_id,
      event.user_id,
      event.anonymous_id,
      event.page_url,
      event.page_path,
      event.page_title,
      event.page_referrer,
      event.device_type,
      event.browser_name,
      event.os_name,
      event.utm_source,
      event.utm_medium,
      event.utm_campaign,
      JSON.stringify(event), // Store full event as JSONB
    ]
  );

  res.status(200).json({ success: true });
} catch (error) {
  console.error('Error ingesting event:', error);
  res.status(500).json({ error: 'Failed to ingest event' });
}
});

// Helper: Hash IP for privacy
function hashIP(ip) {
  return crypto.createHash('sha256').update(ip).digest('hex').substring(0, 16);
}

app.listen(3001, () => {

  console.log('Event API running on port 3001');
});
```

Phase 3: Shopify Integration (Week 3-4)

6.7 Sync Products from Shopify

backend/shopify-sync.js:

```javascript
import fetch from 'node-fetch';
import pg from 'pg';
```

```
const { Pool } = pg;

const pool = new Pool({
  host: process.env.DB_HOST,
  database: process.env.DB_NAME,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
});

const SHOPIFY_DOMAIN = process.env.SHOPIFY_DOMAIN;
const SHOPIFY_ACCESS_TOKEN = process.env.SHOPIFY_ACCESS_TOKEN;

async function syncProducts() {
  const products = await fetchShopifyProducts();

  for (const product of products) {
    await pool.query(
      `INSERT INTO products (
        product_id, product_name, product_handle, product_type,
        price, compare_at_price, in_stock, shopify_updated_at
      ) VALUES ($1, $2, $3, $4, $5, $6, $7, $8)
      ON CONFLICT (product_id) DO UPDATE SET
        product_name = EXCLUDED.product_name,
        price = EXCLUDED.price,
        shopify_updated_at = EXCLUDED.shopify_updated_at`,
      [
        product.id.toString(),
        product.title,
        product.handle,
        product.product_type,
        product.variants[0].price,
        product.variants[0].compare_at_price,
        product.variants[0].inventory_quantity > 0,

        product.updated_at,
      ]
    );
  }

  console.log(`Synced ${products.length} products`);
}

async function fetchShopifyProducts() {
  const response = await fetch(
    `https://${SHOPIFY_DOMAIN}/admin/api/2024-01/products.json`,
    {
      headers: {
        'X-Shopify-Access-Token': SHOPIFY_ACCESS_TOKEN,
```

```
      'X-Shopify-Access-Token': SHOPIFY_ACCESS_TOKEN,
    },
  }
);
const data = await response.json();
return data.products;
}

// Run sync every hour
setInterval(syncProducts, 60 * 60 * 1000);
syncProducts(); // Initial sync
```

Phase 4: Meta Pixel Integration (Week 4)

6.8 Add Meta Pixel to React App

public/index.html:

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <!-- Meta Pixel Code -->
    <script>
      !function(f,b,e,v,n,t,s)
      {if(f.fbq)return;n=f.fbq=function(){n.callMethod?
      n.callMethod.apply(n,arguments):n.queue.push(arguments)};
      if(!f._fbq)f._fbq=n;n.push=n;n.loaded=!0;n.version='2.0';
      n.queue=[];t=b.createElement(e);t.async=!0;
      t.src=v;s=b.getElementsByTagName(e)[0];
      s.parentNode.insertBefore(t,s)}(window, document,'script',
```